

Содержание



Присоединяйтесь к нашему сообществу, чтобы начать свой путь в сфере кибербезопасности уже сегодня!

[Узнать больше](#)

Шпаргалка по Tmux для специалистов по кибербезопасности



Tmux — сокращение от «терминальный мультиплексор» — позволяет запускать несколько сеансов терминала и управлять ими из одного окна, не полагаясь на графический интерфейс. Для большинства пользователей это повышает продуктивность. Для специалистов по информационной безопасности это скорее инструмент выживания, и именно поэтому мы решили создать шпаргалку по Tmux.

Взламывающие Петли



реагирования на инцидент или
потеря сеанса терминала может
активной обратной оболочки,

активного SSH-соединения с
промежуточным хостом. Tmux решает эту проблему, отделяя сеанс терминала от
отображающего его соединения. Вы можете выйти из сеанса, закрыть ноутбук,
подключиться к нему из другого места и продолжить с того места, на котором
остановились.

В этой шпаргалке представлен полный список команд tmux — установка,
сеансы, окна, панели, режим копирования и настройка — с подробным
описанием того, как каждая из них применяется в реальной работе по
обеспечению безопасности: управление одновременными SSH-подключениями
в рамках одного задания, синхронизация команд на нескольких хостах, захват
вывода терминала для составления отчетов и удержание позиции на целевом
устройстве без использования постоянного графического интерфейса, который
может выдать вас.

Установите Tmux

Tmux предустановлен в большинстве дистрибутивов, ориентированных на
безопасность, включая популярные Kali Linux и ParrotOS. Если вы работаете в
системе общего назначения Linux или macOS, установите его одним из
следующих способов:

Команда	Описание
<code>sudo apt-get update && sudo apt-get install tmux</code>	Установка на Debian/Ubuntu/Kali
<code>sudo dnf -y install tmux</code>	Установка на Fedora
<code>sudo yum -y install tmux</code>	Установка на CentOS

Взламывающие Петли		Установить на macOS
		Проверьте установленную версию

Совет: Перед началом работы проверьте свою версию — в старых сборках tmux (до версии 3.0) отсутствуют некоторые функции синхронизации и форматирования панелей, о которых говорится далее в этом руководстве.

Терминология tmux

Термин	Описание	Актуальность для безопасности
Панель	Один псевдотерминал, который может занимать все окно или располагаться рядом с другими	Для каждого инструмента выделена отдельная панель: вывод nmap на одной панели, прослушиватель на другой
Окно	Постоянная коллекция окон, управляемых tmux; сохраняется при отключении	Ваш рабочий контекст для выполнения задачи — сохраняется при обрыве SSH-соединения, кратковременном отключении VPN или закрытии крышки ноутбука
Клиент	Процесс отображения сеанса для вас	Любой терминал, подключенный к сеансу, в том числе терминал на другом компьютере, к которому можно подключиться позже
Сервер	Управляет всеми сеансами; взаимодействует с	В общих или многопользовательских системах сеансы можно привязать к пользовательскому сокету (<code>-S</code>),

Справочник по Tmux: быстрые сочетания клавиш

Запуск из командной строки (вне Tmux):

Команда	Описание
tmux	Запуск tmux
tmux -V	Проверьте версию tmux
tmux ls	Показывать все сеансы
tmux info	Показать все сеансы, окна и панели
tmux a	(Повторное) подключение к последнему созданному сеансу
tmux list-keys less	Перечислите все привязки клавиш
man tmux	Full manual

Run inside a tmux session (prefix key is **Ctrl+b** by default):

Command	Description
Ctrl+b ? then q	List all commands, then close the list
Ctrl+b :	Enter command mode
Ctrl+b l (lowercase l)	Clear console contents

Взламывающие Петли

 computer time
 new window

Ctrl+b ,	Rename current window
Ctrl+b p / Ctrl+b n	Previous/next window
Ctrl+b w	List windows
Ctrl+b %	Split pane vertically
Ctrl+b "	Split pane horizontally
Ctrl+b (arrow keys)	Move between panes
Ctrl+b d	Detach from session

Session Management

Create a session

Command	Description
tmux / tmux new / tmux new-session	Create a new session
Ctrl+b :new	Create a new session from inside tmux
tmux new -s recon	Create a named session (e.g., recon)
Ctrl+b :new recon	Create a new named session from inside tmux

Взламывающие Петли

sessions per client or per target rather than per IP address.  `tmux new -s hackingloops-
On a share or long-running box, this avoids confusion between engagements and makes cleanup at close-out far less error-prone.`

List Sessions

Command	Description
<code>tmux ls / tmux list=sessions</code>	Show all sessions
<code>Ctrl+b s</code>	Show all sessions from inside tmux

Attach, Detach, and Navigate Sessions

Command	Description
<code>tmux a / tmux at / tmux attach</code>	Attach to the most recently created session
<code>tmux a -t hackingloops-extnet</code>	Attach a specific named session
<code>Ctrl+b \$</code>	Rename current session
<code>Ctrl+b d</code>	Detach from session
<code>Ctrl+b w</code>	Session and window preview
<code>Ctrl+b (/ Ctrl+b)</code>	Move to previous/next session

Kill Sessions

Command	Description

Взламывающие Петли



	Kill the last active session
<code>extnet</code>	Kill a specific named session
<code>tmux kill-ses -a</code>	Kill all sessions except the current one
<code>tmux kill-ses -a -t hackingloops-extnet</code>	Kill all sessions except the one named

OPSEC note: At the close of an engagement – especially on a jump host, shared team box, or any system that isn't yours long term – kill all your sessions explicitly rather than just closing the terminal. A detached session (and its socket) will otherwise persist indefinitely, potentially exposing tool output, credentials in scrollback, or the fact you were there at all.

Isolated Sockets (custom -S)

Command	Description
<code>tmux -S /tmp/.eng01 new -s recon</code>	Start a session on a custom socket path
<code>tmux -S /tmp/.eng01 a</code>	Attach using that same custom socket

Using a non-default socket path keeps your sessions separate from other users' tmux sessions on a shared system, and from the default `tmux ls` output of anyone else on the box.

Window Management

Взламывающие
Петли

tion a new session and name the first	
nmap	window
Ctrl+b c	Create new window
Ctrl+b ,	Rename active window
Ctrl+b &	Close active window (confirms with y/n)
Ctrl+b p / Ctrl+b n	Previous / next window
Ctrl+b w	List windows (select/expand with arrow keys)
Ctrl+b :swapw -s 0 -t 2	Swap windows 0 and 2
Ctrl+b :swapw -t -1	Move active windows left by one position

Tool-per-window convention: Name each window after what's running in it – [nmap](#) , [burp](#) , msf , notes , loot – rather than leaving default numeric labels. This makes a session self-documenting at a glance, which matters both for your own workflow across a multi-day engagement and if a teammate ever needs to pick up where you left off.

Pane Management

Command	Description
Ctrl+b q	Show pane numbers briefly
Ctrl+b q then 0-9	Switch to pane by number

Взламывающие Петли

≡ `xt pane`
`'out of the current pane`

<code>Ctrl+b !</code>	Convert current pane into a new window
<code>Ctrl+b %</code>	Split vertically
<code>Ctrl+b "</code>	Split horizontally
<code>Ctrl+b</code> (arrow keys)	Switch to adjacent pane
<code>Ctrl+b ;</code>	Toggle last active pane
<code>Ctrl+b [spacebar]</code>	Toggle between pane layouts
<code>Ctrl+b { / Ctrl+b }</code>	Move current pane left/right
<code>Ctrl+b x</code>	Close current pane
<code>Ctrl+b :resize-pane -L 5</code>	Resize pane boundary (flags: <code>-U / -D / -L / -R</code>)

Synchronize Panes

Command	Description
<code>Ctrl+b :setw synchronize-panes on</code>	Send every keystroke on all panes in the window simultaneously
<code>Ctrl+b :setw syncrhonize-panes off</code>	Turn synchronization back off

This is one of the most useful tmux features for security work – see the dedicated section below for a full walkthrough.

Взламывающие
Петли

≡ i number:

Combo	Layout
Meta 1	Uniform vertical split
Meta 2	Uniform horizontal split
Meta 3	Horizontal split, main pane on top
Meta 4	Vertical split, main pane on left
Meta 5	Tiled, new panes bottom-then-right

Copy Mode & Buffers

Tmux’s buffer is its internal clipboard. Buffers are numbered (`buffer_0` , `buffer_1` , ...) in the order they’re created.

Command	Description
<code>Ctrl+b :setw -g mode-keys vi</code>	Use vi-style keys in copy mode
<code>Ctrl+b [</code>	Enter copy mode
<code>↑ / ↓</code>	Scroll up/down
<code>q</code>	Quit copy mode
<code>0 / \$</code>	Beginning / end of line

Взламывающие Петли

≡ t/last line
ve left/down/up/right

b / e / w	Word backward / word-end forward / word forward
/ / ?	Search forward/backward
n / N	Next/previous search
[spacebar]	Begin selection
[Enter]	Copy selection
Esc	Clear selection
Ctrl+b	Paste selection
Ctrl+b :show-buffer	Display buffer_0 contents
Ctrl+b :capture-pane	Copy the pane's visible contents to a buffer
Ctrl+b : list-buffers	Show all buffers
Ctrl+b :choose-buffer	Show all buffers and paste to a chosen one
Ctrl+b :save-buffer findings.txt	Save buffer contents to a file
Ctrl+b :delete-buffer -b 1	Delete buffer_1

Взламывающие
Петли

manually setting up a separate logging tool.

mbined with `save-buffer` is a fast,
put for a report – e.g., capturing an
tation sequence straight to a text file

Tmux Configuration

Tmux is configured via `~/.tmux.conf`. Create it if it doesn't exist:

```
touch ~/.tmux.conf
```

Reload after editing:

```
tmux source-file ~/.tmux.conf
```

Tmux Keybindings

Config Line	Effect
<code>unbind '"' bind - split-window -v</code>	Rebind horizontal split from " to –
<code>unbind % bind split-window -h</code>	Rebind vertical splits from % to
<code>unbind C-b set -g prefix C-a</code>	Change prefix key from Ctrl+b to Ctrl+a

Why this matters for security work: Some CLI tools and TUI-based utilities (certain vim configurations, some CTF/exploit-dev tooling) use `Ctrl+b` for their own bindings.

Взламывающие Петли

commonly claimed – `Ctrl+a` is a
≡ when you're running tmux inside tmux, or

Default to vi Keys in Copy Mode

```
setw -g mode-keys vi
```

Status Bar Customization

Config Line	Effect
<pre>set -g status-justify [left/centre/right]</pre>	Align window names in the status bar
<pre>set -g status-left '...'</pre>	Replace the session name shown in the status bar
<pre>setw -g window-status-format '# [fg=white,bg=black]#I'</pre>	Custom window name formatting

Useful variables: `#I` (window index), `#S` (session name), `#W` (window name). Full formatting reference: `man tmux`, then search for the `STYLES` section.

To reset to defaults: clear `~/.tmux.conf` and run `tmux kill-server`, then restart your terminal.

Using Tmux with SSH

A common security workflow is managing several simultaneous SSH connections – to different hosts in scope, jump boxes, or compromised systems – without any one of them monopolizing your terminal.

Взламывающие Петли



low per target: `tmux new -s`

`engagement -n host1`

2. In each window, open an SSH connection: `ssh user@10.10.10.11`
3. Create additional windows (`Ctrl+b c`) for additional hosts, renaming each (`Ctrl+b ,`) to match the target
4. Detach (`Ctrl+b d`) when you need your terminal back – every SSH connection stays alive in the background
5. Reattach later (`tmux a -t engagement`) to resume exactly where you left off.

This means a dropped VPN connection, a closed laptop lid, or a network hiccup doesn't kill your SSH sessions – only your view of them is interrupted, not the session themselves.

Synchronize-Panes Across Hosts

`synchronize-panes` sends every keystroke you type to **every pane in the current window** at once. Applied to a window where each pane holds an SSH session to a different host, this becomes a lightweight way to run the same command across many systems simultaneously.

Example workflow:

1. Open a window and split it into multiple panes (`Ctrl+b % / Ctrl+b "`)
2. SSH into a different host from each pane
3. Enable sync: `Ctrl+b :setw synchronize-panes on`
4. Type a command once – it executes on every connected host at the same time
5. Disable sync (`Ctrl+b :setw synchronize-panes off`) before doing anything host-specific, to avoid accidentally running a targeted command everywhere.

Where this is useful in practice:

Взламывающие Петли

| command across a batch of in-scope



ettings across a fleet during an

assessment

- Applying identical remediation or verification commands across multiple systems during incident response

Caution: Because every keystroke goes to every pane, this is also an easy way to run a destructive or host-specific command somewhere you didn't intend to. Always double-check which panes are active before typing, and turn sync off as soon as you're done with the batch operation.

Detach & Reattach: Surviving Dropped Connections

This is the core reason Tmux earns a permanent place in a security professional's toolkit.

The Mechanic

Command	Description
<code>Ctrl+b d</code>	Detach from current session
<code>tmux ls</code>	Confirm the session is still running
<code>tmux a -t [session]</code>	Reattach to it

Why This Matters Operationally

Long-running jobs survive disconnection. A password-cracking job (hashcat, hydra), a long nmap scan, or a data exfiltration transfer keeps running inside the tmux session even in your local terminal, SSH client, or network connection drops.

Reverse shells and footholds persist. If you've caught a reverse shell inside a tmux window on your attack box, losing your local terminal doesn't lose the shell – reattach

Взламывающие Петли



Because tmux sessions live on the server and the tmux process is running on (not on your local client), you can detach from your desktop and reattach from a laptop elsewhere, as long as both can reach the same session – useful for handing off or resuming an engagement from a different location.

Unexpected disconnects don't cost you the session. A closed laptop lid, a Wi-Fi drop, or an SSH timeout kills the *connection*, not the *session*. Reattaching picks up exactly where you left off, scrollback and all.

Security Tradecraft & OPSEC Notes

A few additional practices worth knowing, specific to how tmux gets used in offensive and defensive security work:

- **Persistent shells on a target.** Running tmux on a system you've gained access to – inside an existing shell – lets you detach and reattach to that shell later without re-establishing access, similar to how it's used on your own attack box. This is well-known tradecraft on both red and blue teams; defenders should be aware that an unexpected tmux/screen process or socket file under `/tmp` can be an indicator of persistence.
- **Continuous pane logging (pipe-pane).** `Ctrl+b :pipe-pane -o 'cat >> ~/session.log` pipes all output to a log file in real time, useful for maintaining an audit trail during an engagement or investigation without manually capturing panes.
- **Custom sockets for isolation.** As noted earlier, `tmux -S /path/to/socket` keeps a session off the default `tmux ls` listing for other users on a shared system.
- **Clean up after yourself.** Kill sessions (`tmux kill-ses`) and remove custom socket files at the end of an engagement, especially on any system that isn't exclusively yours. Leftover sessions can expose command history, tool output, or credentials to anyone else with access to that box.



manage multiple terminal sessions

from one window, and – critically for security work – keeps those sessions running even after you disconnect from them.

How do I enter commands in tmux?

Either directly from the CLI prefixed with `tmux`, or inside a session using the prefix key **Ctrl+b** for short commands, or **Ctrl+b :** for longer named commands.

How do I create tmux sessions?

`tmux new -s <name>` – see Session Management

How do I list all tmux sessions?

`tmux ls` – see Session Management

How do I switch between windows in tmux?

`Ctrl+b n` (next) or `Ctrl+b p` (previous) inside a session – see Window Management

How do I use tmux with SSH?

Open a tmux session, dedicate a window to each SSH connection, detach when you need your terminal back – the connections stay alive. See Using Tmux with SSH

Why is tmux especially useful in security work?

It lets you run parallel operations – multiple SSH sessions, long-running scans or cracking jobs, reverse shells – without losing any of them to a dropped connection, closed laptop, or network interruption. It also supports running identical commands across multiple hosts at once via pane synchronization.

How do I recover a session after a dropped connection?

Run `tmux ls` to confirm the session is still alive, then `tmux a -t <session>` to reattach. See Detach & Reattach



This was created for those who want to obtain a lucrative career in cybersecurity. Get this free guide delivered to your email that will show you step-by-step how to get into cybersecurity.

Quick Link

[Home](#)

[About Us](#)

[Membership](#)

[Blog](#)

[Contact Us](#)

Contact Detail

500 Westover Dr #8208, Sanford, NC 27330

hackingloopsteam@hackingloops.com